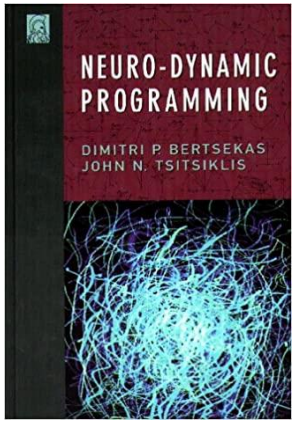
IE540
L8. Deep Q Network

KAIST ISE
Hayong Shin

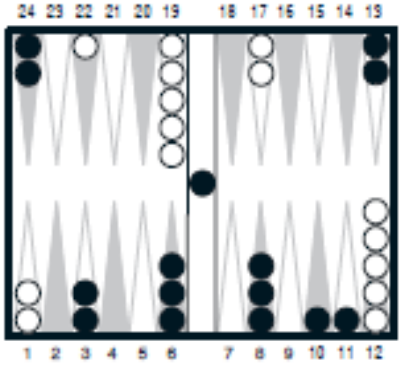
Overview

- Model is known → DP (in Lecture 3)
  - Evaluation (prediction) : Iterative policy evaluation
  - Control (finding optimal solution)
    - PI → GPI → VI
    - Async. DP
- Model is unknown → RL (by use of interaction samples)
  - Value based RL
    - MC method : Evaluation, Control (in Lecture 4)
    - TD method : Q, SARSA (in Lecture 5)
    - n-step bootstrapping (in Lecture 6)
    - value function approximation (in Lecture 7)
    - NN, DQN (Lecture 8)
  - Policy based RL (Lecture 9~11)
    - Policy gradient
    - Derivative free methods

Function approximation



- Representation of $Q(s,a)$ function
  - Table lookup function
    - Difficult to handle : large state-action space
  - Function approximation $q(s,a;w) \approx Q(s,a)$
    - Linear FA $q(s,a;w) = w^T x(s,a)$
      - $x(s,a)$: feature vector
    - Neural net for more complex Q function
      - NN : a universal function approximator
      - Neuro-Dynamic Programming
        - » by Bertsekas & Tsitsiklis (MIT ORC), 1996
      - TD-Gammon, 2002 : using NN for backgammon
      - Auto-gradient available through back-propagation
        - » TensorFlow
        - » PyTorch
      - Explosion of deep learning : CNN (AlexNet), 2012
      - DRL by DeepMind, Nature 2015
        - » DL+RL → DQN for Atari games



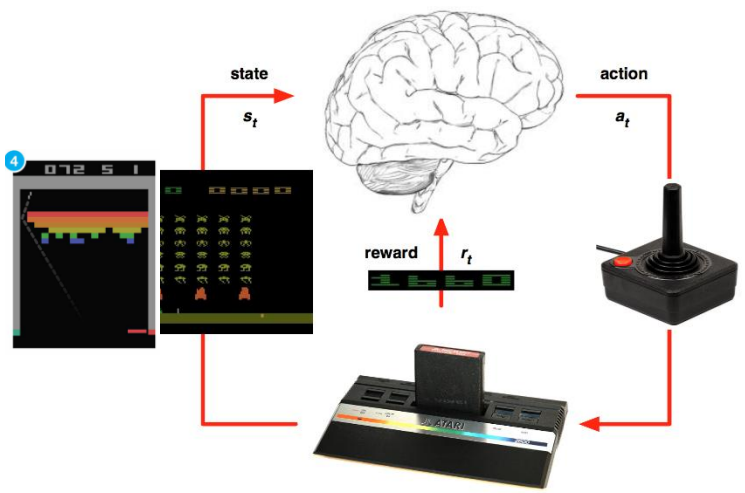
Google Deepmind

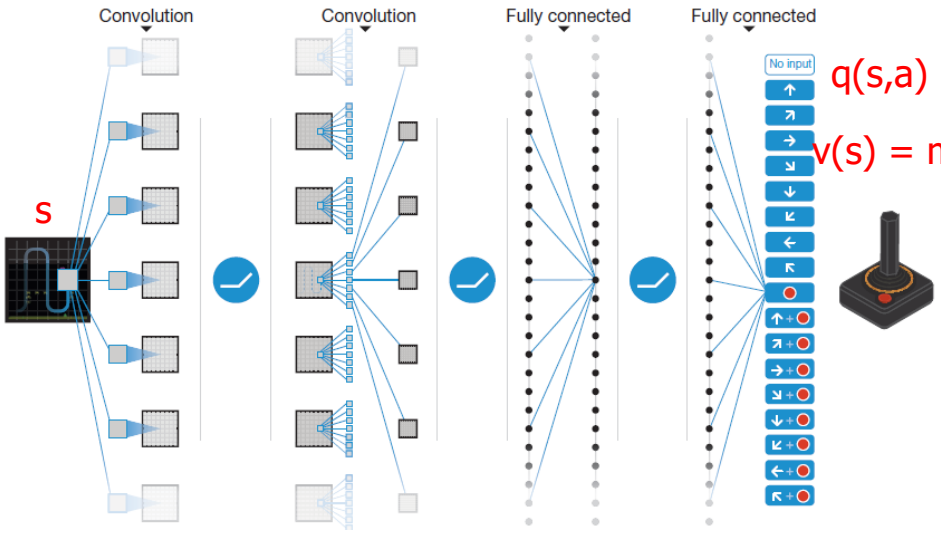
- In search of AGI : Artificial General Intelligence, like human
  - Instead of Narrow AI (hand-crafted software only for specific task)
    - Deep Blue, which defeated human champion of chess
- 2015 Nature paper : Playing 49 Atari video games like human
  - DRL (Deep Reinforcement Learning)
    - **Reinforcement learning** framework

    + **Deep learning** for approximating value function

  - Without any prior knowledge about the games

  → Huge success in almost all games





DQN

Human-level control through deep reinforcement learning

Volodymyr Mnih[1]*, Koray Kavukcuoglu[1]*, David Silver[1]*, Andrei A. Rusu[1], Joel Veness[1], Marc G. Bellemare[1], Alex Graves[1], Martin Riedmiller[1], Andreas K. Fidjeland[1], Georg Ostrovski[1], Stig Petersen[1], Charles Beattie[1], Amir Sadik[1], Ioannis Antonoglou[1], Helen King[1], Dharshan Kumaran[1], Daan Wierstra[1], Shane Legg[1] & Demis Hassabis[1]



Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^{-} = \theta$

For episode $= 1, M$ **do**

  Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

  For $t = 1$,T **do**

    With probability ε select a random action a_t

    otherwise select $a_t = \text{argmax}_a Q(\phi(s_t), a; \theta)$

    Execute action a_t in emulator and observe reward r_t and image x_{t+1}

    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

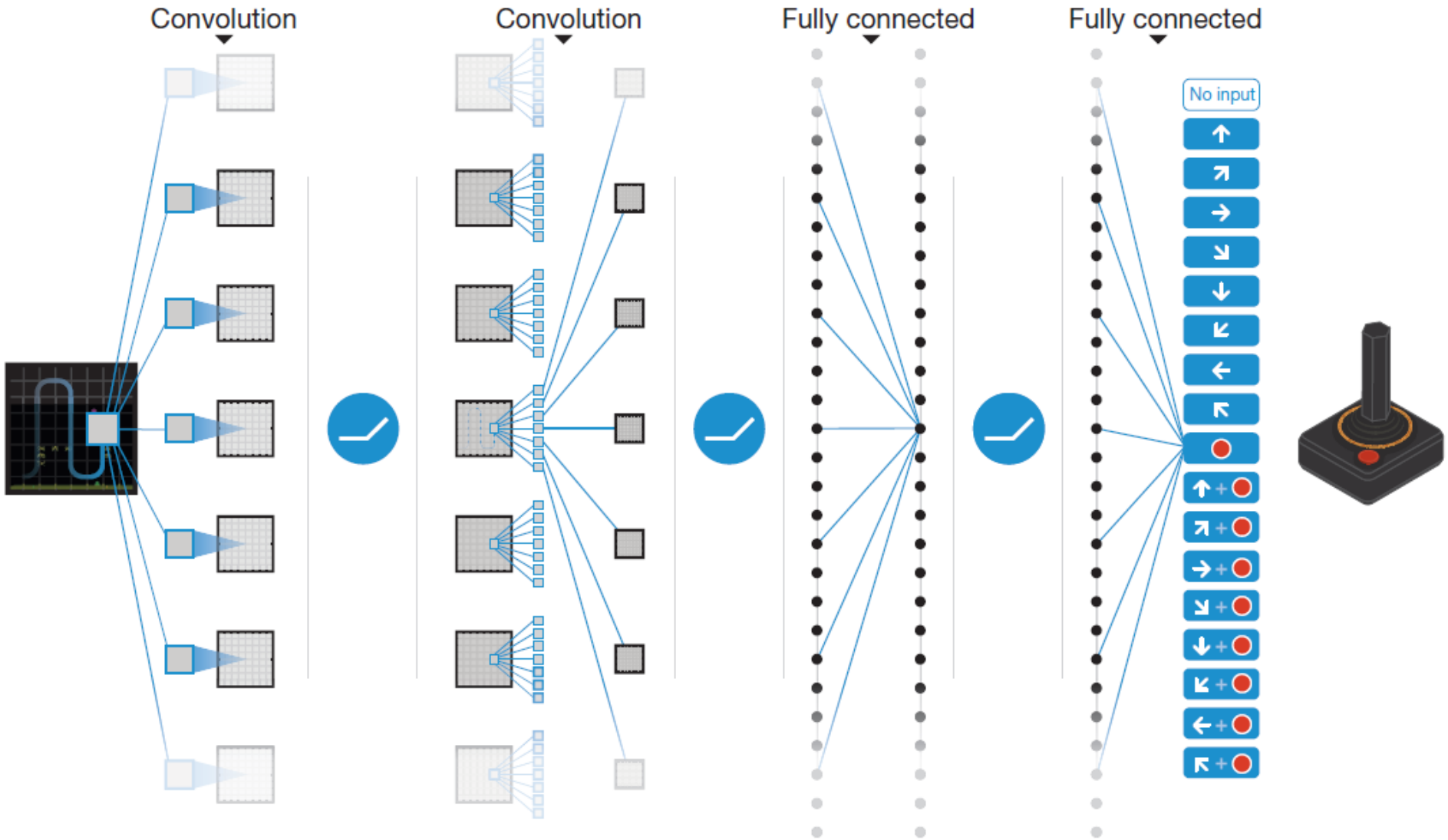
    Set $y_j = \begin{cases} r_j & \text{if episode terminates at step j+1} \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^{-}) & \text{otherwise} \end{cases}$

    Perform a gradient descent step on $\left(y_j - Q(\phi_j, a_j; \theta)\right)^2$ with respect to the network parameters θ

    Every C steps reset $\hat{Q} = Q$

  End For

End For

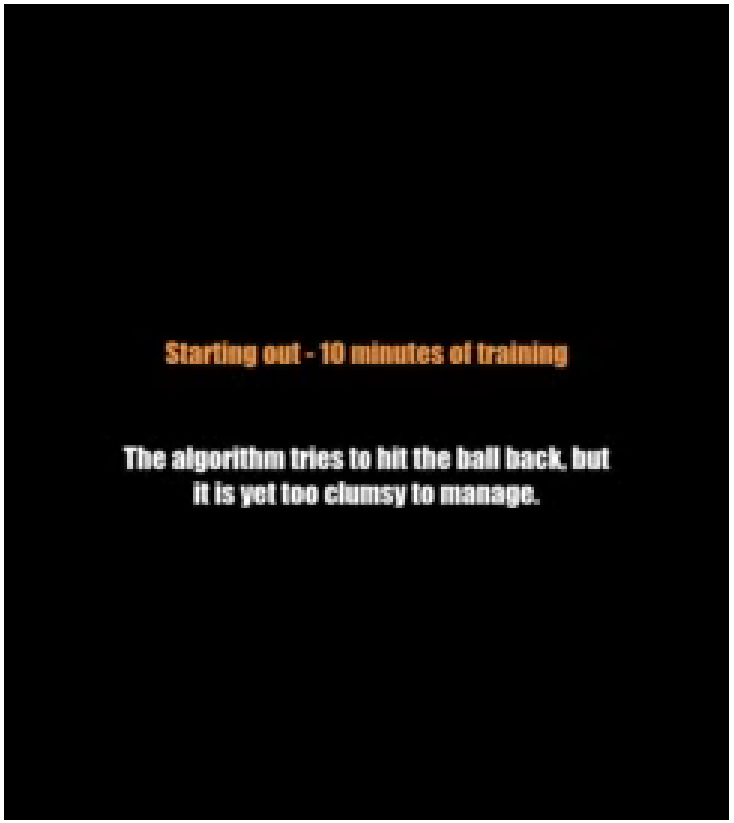


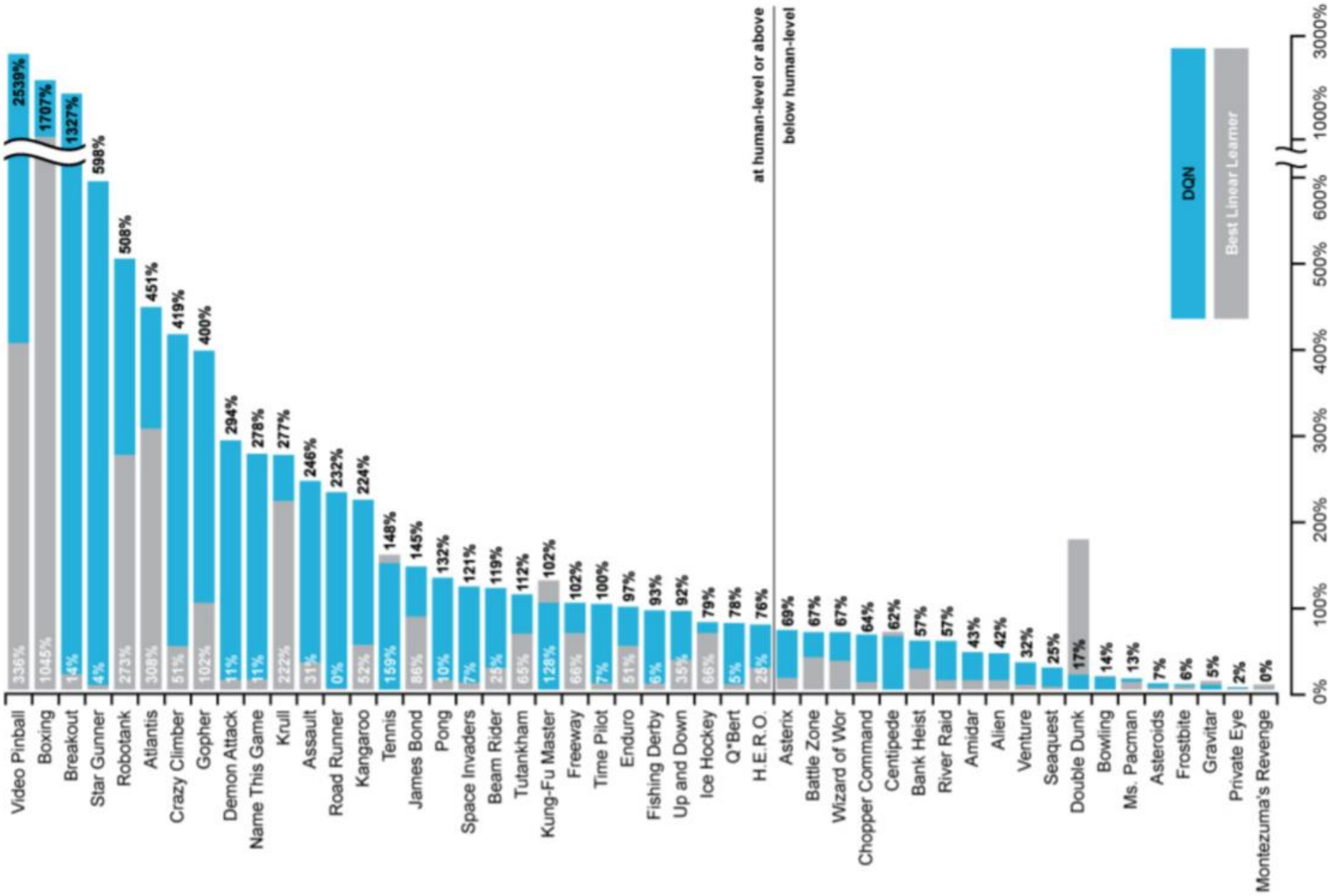
D. Silver, DQN, Nature 2015

Details of DQN Atari game

- Input : sequence of screen images
  - Original image : 210x160, 128 colors
    → 84x84 grayscale image
  - Stack 4 most recent images

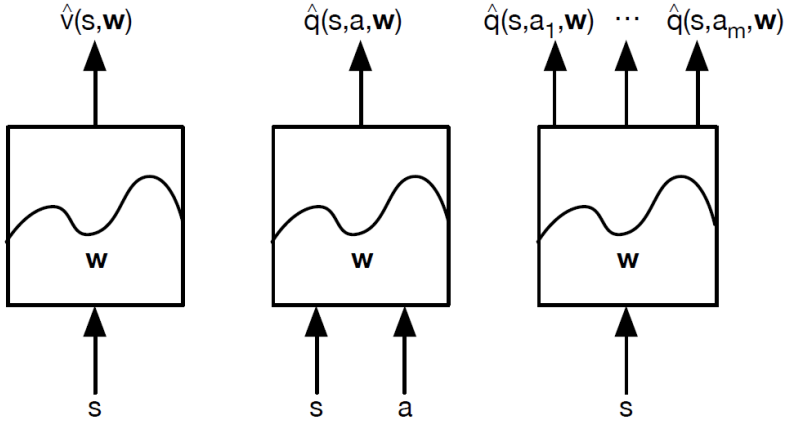
  → 84x84x4 images
- Network architecture : 3 conv + 2 FC
  - Conv. & downsample → 20x20x32
  - Conv. & downsample → 9x9x64
  - Conv. → 7x7x64 → flatten to 3136
  - FC → 512 hidden units
  - FC → 18 outputs (action values)
- Experience replay
  - Mini-batch w/ RMSProp
- Reward : game score change (-1, 0, +1)
- ε-greedy (decreasing as 1/n)
- Fixed Q (fixed weight)
- TD error clipping (-1~+1)





D. Silver, DQN, Nature 2015

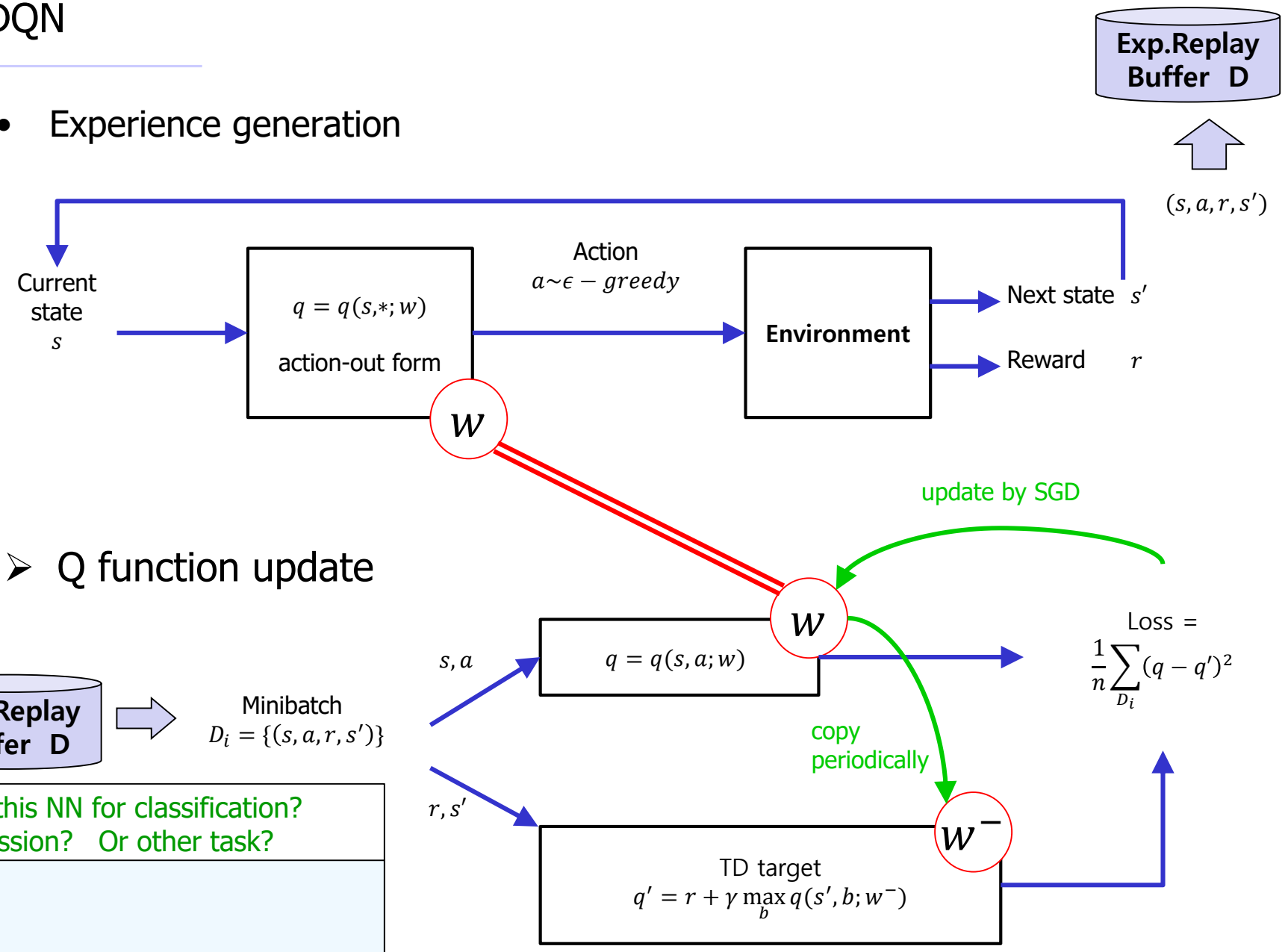
Innovations in DQN



- DRL : DL + RL
  - Q-learning : TD(0)
  - neural net for Q function : action-out
- Batch RL : Replay buffer
  - D = set of { (s, a, r, s') }
  - sample efficiency
  - break correlation between successive states
  - mini-batch → SGD
    - D_i = random subset of replay buffer D
- Fixed parameter for TD target
  - TD target : $q' = r + \gamma \max_b q(s', b; w^-)$
  - more stable convergence

DQN

- Experience generation



➢ Q function update

Exp.Replay Buffer D → Minibatch $D_i = \{(s, a, r, s')\}$

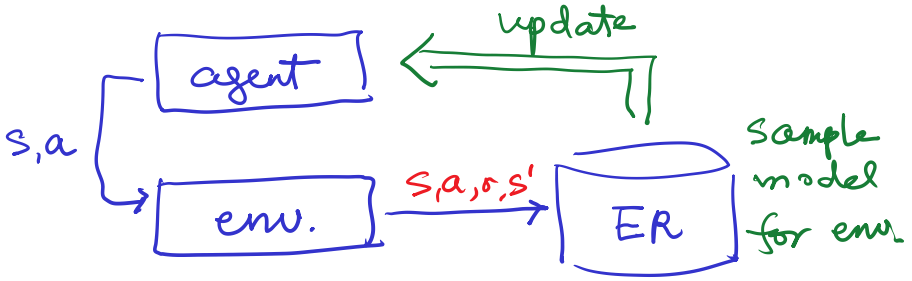
TAQ. Is this NN for classification? Or regression? Or other task?

Experience Replay



- Decorrelate successive state
- ER ≈ Model for environment
  - state transition : $p(s'|s,a)$
  - reward : $p(r|s,a)$
  - state distribution in the past : $\rho(s)$

  → used for computing the TD error



- Implementing ER
  - Circular queue with fixed capacity (FIFO)
  - ER contains more recent experiences

TAQ. Why are recent experience segments more important ?

TAQ. Then, how about assigning bigger probability to recent segments.

Game	Linear	Deep Network	DQN w/ fixed Q	DQN w/ replay	DQN w/replay and fixed Q
Breakout	3	3	10	241	317
Enduro	62	29	141	831	1006
River Raid	2345	1453	2868	4102	7447
Seaquest	656	275	1003	823	2894
Space Invaders	301	302	373	826	1089

DQN for Cartpole

```
import ...


class ReplayMemory(object):

    def __init__(self, capacity):...

    def segTuple(self, s, a, r, sp, done):...

    def push(self, s, a, r, sp, done):...

    def sample(self, batch_size):...
```

```
class MLP(torch.nn.Module):
    def __init__(self, lsize):
        super().__init__()
        self.layers = nn.ModuleList()
        self.n_layers = len(lsize) - 1
        for i in range(self.n_layers):
            self.layers.append(torch.nn.Linear(lsize[i], lsize[i+1]))
            # self.layers.append(nn.BatchNorm2d(lsize[i]))

    def forward(self, x):
        x = x.reshape(x.size(0), -1)
        for i in range(self.n_layers):
            x = self.layers[i](x)
            if i < self.n_layers-1:
                # x = torch.tanh(x)
                x = F.relu(x)
        return x  # softmax is done within F.cross_entropy
```

DQN for Cartpole

skeleton of this code will be provided as the starting point of homework assignment.

```
class AgentBase:
    def __init__(self, env):...

    def getStateRep(self, obs):...

    def getQ(self, s):...

    def piGreedy(self, state):...

    def piEpsGreedy(self, state):...

    @staticmethod
    def getSoftmax(z, tau=1):...

    def piSoftmax(self, state):...

    def trainBatch(self, batch):...

    def trainFromRB(self):...

    def runEpisode1(self, saveRB=True, maxStep=200):...

    def runTest(self, nEpisode=1, maxStep=1000):...

    def runMany(self, nEpisode=1000, maxStep=200):...
```

```
class AgentNN(AgentBase):
    def __init__(self, env):
        super().__init__(env)
        self.prepareNN()

    def prepareNN(self):...

    def updateTarget(self):...

    def getStateRep(self, s):...

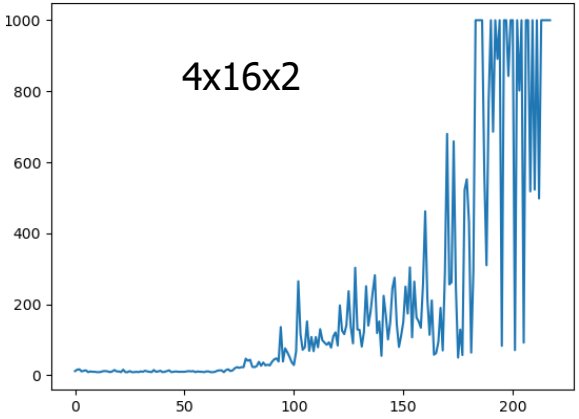
    def getQ(self, stateTensor):
        s = stateTensor.to(self.device)
        with torch.no_grad():
            q = self.qf(s)
        q = q.cpu().numpy()
        return q

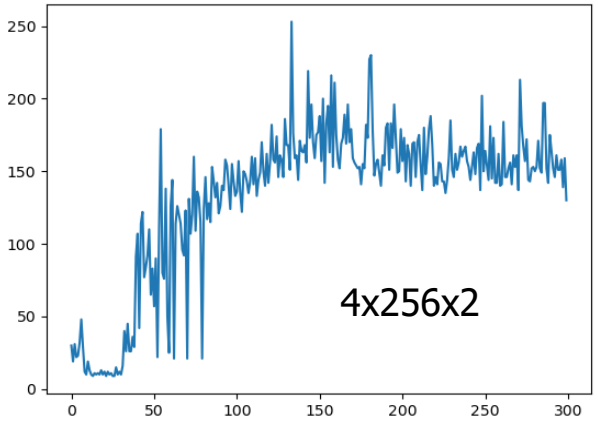
    ...
    def trainBatch(self, batch):...

    def load(self, fn):...
    def save(self, fn):...
```

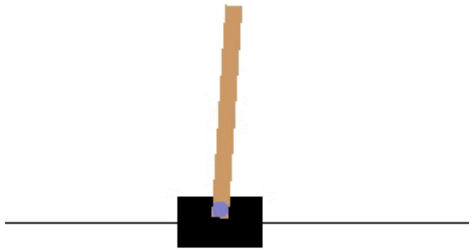
Training DQN for Cartpole

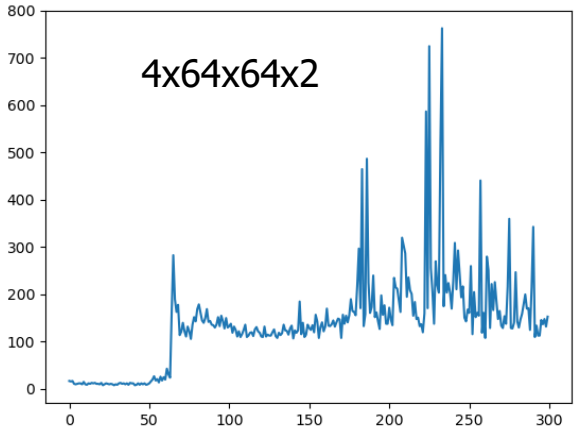
- Not easy to get stable training
  - Q function is rather complicated

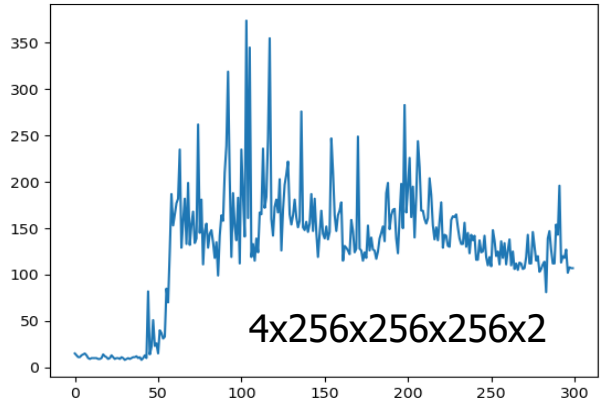


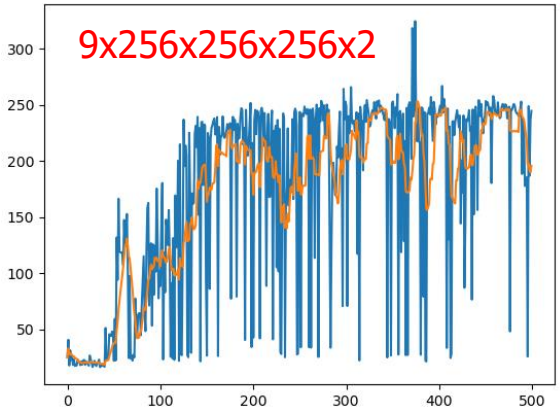


9 input:
Raw state(4)
+5 quadratic terms
9x256x256x256x2









Improvement since Nature DQN (2015)

- Double DQN (Hasselt, Guez, Silver, "DRL with double Q learning", AAAI 2016)
  - Double Q learning applied to DQN
    - mitigate maximization bias by maintaining two Q networks

- Prioritized experience replay
  - cf. prioritized sweeping in DP(value iteration)
  - update w using experience segment (s, a, r, s') with bigger TD error

- Dueling network : split Q network into 2 channels
  - $q(s,a) = V(s) + A(s,a)$
    - state value function: $V(s)$, action independent part
    - action advantage function: $A(s,a)$, action dependent part
  - learn $V(s)$ and $A(s,a)$ separately in a single network → dueling architecture
    - more natural representation of value

- Rainbow : combining the above → better performance
  - 3x mean score on Atari game compared to Nature DQN

Double DQN[2]

- Double Q learning[1]
  - mitigate maximization bias
  - by using 2 sets of Q function
    - one for selecting best action
    - the other for evaluating that action

$$Q^A(S,A) \overset{\alpha}{\leftarrow} R + \gamma Q^B\left(S', \underbrace{\arg\max_a Q^A(S',a)}_{\text{select } a \text{ from } Q^A}\right)$$

get value from Q^B

- DQN : $q(s,a;w)$ as an NN (Q-net)
- Double DQN
  - Current Q-net param : w → action selection $a^* = \underset{a'}{\text{argmax}}\, q(s',a';w)$
  - Target Q-net param : w^- →TD target evaluation $r + \gamma q(s',a^*;w^-)$
  - Define loss & update w using SGD from Replay memory
    - $\text{loss} = \frac{1}{|M|}\sum_{(s,a,r,s')\in M}\left(r + \gamma q(s', \underset{a'}{\text{argmax}}\, q(s',a';w); w^-) - q(s,a;w)\right)^2$
    - M : minibatch sampled from replay memory
    - $w \leftarrow w - \alpha\nabla_w \text{loss}$
  - Update target param using periodic copy
    - or exponential smoothing: $w^- \leftarrow \tau w + (1-\tau)w^-$

TAQ. Is more computation required for double DQN?

[1] Hasselt, "Double Q learning", AAAI 2010
[2] Hasselt, Guez, Silver, "DRL with double Q learning", AAAI 2016

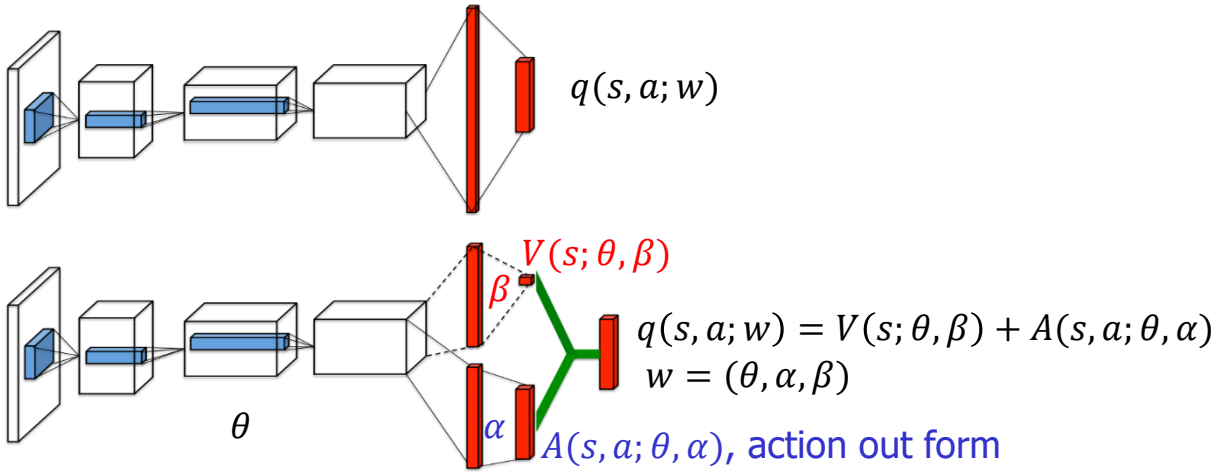
Prioritized Experience Replay[1]

- Recall prioritized sweeping in DP value iteration
  - faster convergence
- Probability of selecting exp. segment (s_i, a_i, r_i, s_{i+1}) based on TD error
  - larger |TD error| → higher probability of selection
    - $\delta_i = r_i + \gamma \max_{a'} Q(s_{i+1}, a'; w^-) - Q(s_i, a_i; w)$
  - priority
    - $p_i = |\delta_i|$, c : hyperparameter → TD-error proportional
    - $p_i = 1/rank(|\delta_i|)$ → rank based
    - selection probability $P(i) = p_i$
  - stochastic prioritization : selection the segment (s_i, a_i, r_i, s_{i+1})
    - with probability $P(i) = \frac{p_i^c}{\sum_k p_k^c}$, c : hyperparameter
      - $c = 0$: uniform, $c = \infty$: greedy
  - new segments are added w/ maximum priority
- For efficient implementation
  - use special data structure "sum tree"
    - similar to binary heap
    - parent node value = left node value + right node value

[1] Saul, Silver et al (Deepmind), "Prioritized Experience Replay", ICLR 2016

Dueling DQN [1]

- Dueling architecture : split $q(s,a;w) = V(s;w) + A(s,a;w)$
  - learn $V(s;w)$ and $A(s,a;w)$ separately in a single network
    - more natural representation of value
      - state value function: $V(s;w)$, action independent part
      - action advantage function: $A(s,a;w)$, action dependent part
    - while sharing features in the lower part of the network
  - need to ensure $V(s;w) = \max_a q(s,a;w)$

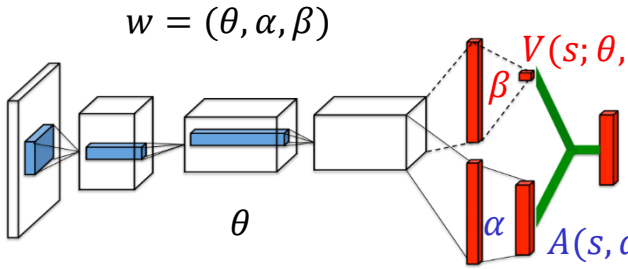


[1] Wang et al (Deepmind), "Dueling network for DRL", ICML 2016 (best paper award)

Dueling DQN

- How to ensure
  - $V(s) = \max_a q(s,a) = V(s) + \max_a A(s,a)$

  → $\max_a A(s,a) = 0$
- when computing $q(s,a;w)$, enforce the above by:
  - option 1 : $q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left\{A(s,a;\theta,\alpha) - \max_{a'} A(s,a';\theta,\alpha)\right\}$
  - option 2: $q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left\{A(s,a;\theta,\alpha) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a';\theta,\alpha)\right\}$
    - a little biased, but more stable in optimization



$$q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left\{A(s,a;\theta,\alpha) - \max_{a'} A(s,a';\theta,\alpha)\right\}$$

$$q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left\{A(s,a;\theta,\alpha) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a';\theta,\alpha)\right\}$$

Rainbow: Combining improvements in DRL

- Combining major extensions
  - Double Q learning
  - Prioritized ER
  - Dueling network
  - Multi-step learning : use n-step target
    - $\sum_{k=0}^{n-1} \gamma^t r_{t+k} + \gamma^n \max_{a'} Q(s_{t+n}, a'; w^-)$
  - Distributional RL
    - Learn Q as a distribution
  - Noisy net
- Integration of extensions

  : Rainbow agent
  - n-dist = n-step + distributional

  + double Q for n-dist.

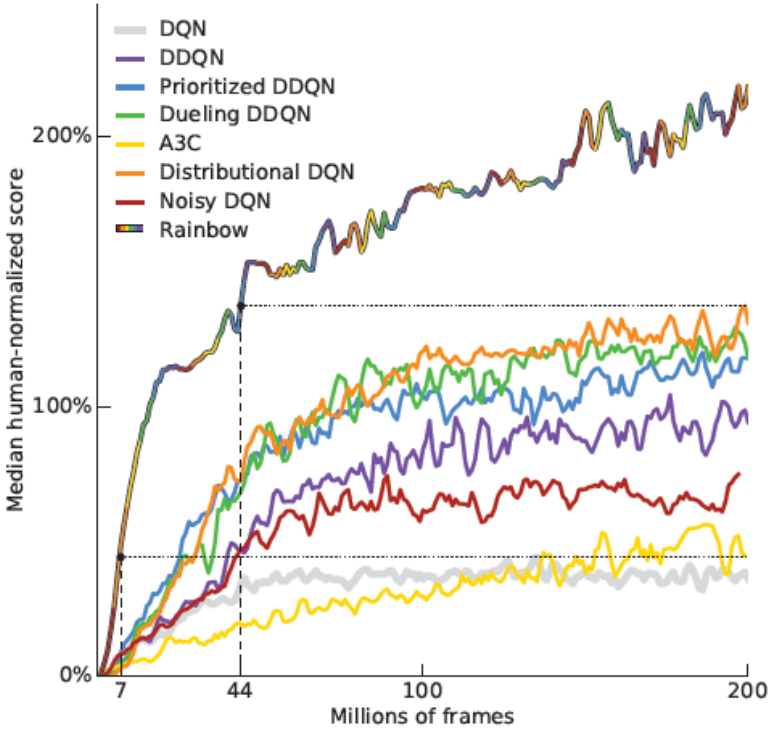
  + PER for n-dist.

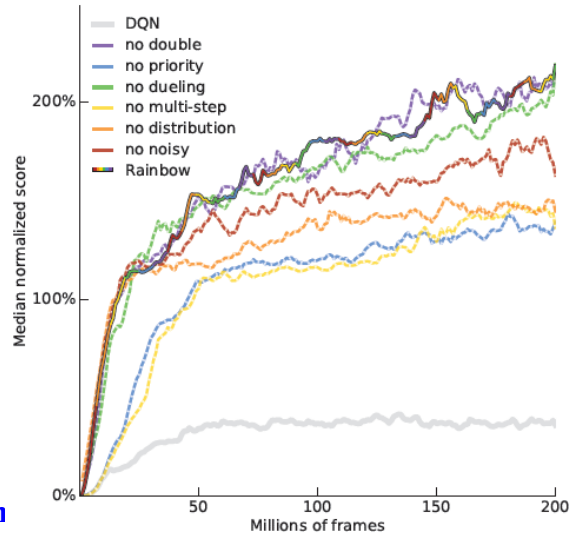
  + dueling net for n-dist.

  + noisy net
- Ablation study (median performance, not always)
  - PER and multi-step learning : most crucial

  > Distributional RL > noisy net

  > Double Q, Dueling net





Hessel et al (Deepmind), "Rainbow: Combining improvements in DRL", AAAI 2018